

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

UCEST 105: ALGORITHMIC THINKING WITH PYTHON LAB

LAB MANUAL

PREPARED BY:

Dr. SANAJ M S

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

- M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.
- M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.
- M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

- M1: To seek and strive for advancements in technological domains.
- M2: To create socially responsible engineers in the field of computer science.
- M3: To bridge the gap between science and mankind through meaningful innovation and service.

Program Educational Objectives (PEOs)

- PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.
- PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE SYLLABUS

LAB Experiments:

1. Demonstrate about Basics of Python Programming
2. Demonstrate about fundamental Data types in Python Programming. (i.e., int, float, complex, bool and string types)
3. Demonstrate different Arithmetic Operations on numbers in Python.
4. Create, concatenate, and print a string and access a sub-string from a given string.
5. Familiarize time and date in various formats (Eg. "Sun May 29 02:26:23 IST 2017")
6. Write a program to create, append, and remove lists in Python using numPy.
7. Programs to find the largest of three numbers.
8. Convert temperatures to and from Celsius, and Fahrenheit. [Formula: $c/5 = f-32/9$]
9. Program to construct the stars (*) pattern, using a nested for loop
10. Program that prints prime numbers less than 20.
11. Program to find the factorial of a number using Recursion.
12. Recursive function to add two positive numbers.
13. Recursive function to multiply two positive numbers

14. Recursive function to the greatest common divisor of two positive numbers.
15. Program that accepts the lengths of three sides of a triangle as inputs. The program output should indicate whether or not the triangle is a right triangle (Recall from the Pythagorean Theorem that in a right triangle, the square of one side equals the sum of the squares of the other two sides). Implement using functions.
16. Program to define a module to find Fibonacci Numbers and import the module to another program.
17. Program to define a module and import a specific function in that module to another program.
18. Program to check whether the given number is a valid mobile number or not using functions?

Program: B. Tech	Department: Computer Science & Engineering
Course: Algorithmic Thinking with Python	Semester: S1 Batch: C Credits: 4
Course Code: UCEST105	Regulation: 2024
Course Area/Domain: Algorithms And Theory of Algorithms	L: 3 T: 0 TW/SL: 3 P: 2
Corresponding Lab Course Code (If Any):	Lab Course Name: Nil

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Utilize computing as a model for solving real-world problems.
CO2	Articulate a problem before attempting to solve it and prepare a clear and accurate model to represent the problem.
CO3	Use effective algorithms to solve the formulated models and translate algorithms into executable programs.
CO4	Interpret the problem-solving strategies, a systematic approach to solving computational problems, and essential Python programming skills

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH												PSO STRENGTH		
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	3									3	3	3	
CO2	3	3	3									3	3	3	
CO3	3	3	3									3	3	3	
CO4	3	3	3									3	3	3	

GENERAL INSTRUCTIONS (for the lab)**Preparation:**

- Ensure you have Python installed on your computer. You can download the latest version from python.org.
- Familiarize yourself with the IDE or text editor you will be using (e.g., PyCharm, Visual Studio Code, Jupyter Notebook, etc.).
- Make sure to have all required libraries or packages installed. You can use pip to install packages (e.g., `pip install numpy`).

Lab Setup:

- Create a new directory for the lab session to keep your work organized.
- Open your IDE or text editor and create a new Python file for each exercise or project.
- Ensure your workspace is properly configured (check for interpreter settings in your IDE).

Documentation:

- Start each script with comments that include your name, date, and a brief description of what the script does.
- Write clear comments throughout your code to explain your thought process and the purpose of each section.

Coding Standards:

- Follow PEP 8 guidelines for Python coding standards (naming conventions, indentation, line length, etc.).
- Use meaningful variable names that convey the purpose of the variable.

Testing and Debugging:

- Test your code frequently as you write it. Use print statements or debugging tools to trace the execution flow and identify issues.
- Handle exceptions using try-except blocks to make your code more robust.

Submission:

- Follow the submission guidelines provided by your instructor (e.g., format, naming conventions).

- Ensure your code runs without errors and meets all the requirements before submission.

Collaboration:

- Collaborate with your peers if allowed, but ensure that your submitted work is your own.
- Discuss different approaches to solving problems and share insights.

Resources:

- Utilize online resources, such as documentation, forums (e.g., Stack Overflow), and tutorials, if you encounter difficulties.
- Refer to the Python Standard Library for built-in functions and modules that may assist in your tasks.

Ask for Help:

- Don't hesitate to ask your instructor or teaching assistants for help if you get stuck or need clarification on the tasks.

BEST PRACTICES**Understand the Basics:**

- Ensure you have a solid grasp of Python basics, including syntax, data types, control structures, functions, and error handling.

Plan Your Code:

- Before you start coding, take time to plan the structure of your program. Outline the steps needed to solve the problem and think about how you will organize your code.

Write Readable Code:

- Follow PEP 8 style guidelines for writing clear and readable code. This includes consistent naming conventions, indentation, and spacing.
- Use comments and docstrings to explain your code's purpose, especially for complex logic or functions.

Use Version Control:

- Utilize version control systems like Git to track changes in your code. This allows you to revert to previous versions and collaborate effectively with others.

Modular Programming:

- Break your code into smaller, reusable functions or modules. This makes your code easier to understand, test, and maintain.
- Each function should have a single responsibility.

Testing:

- Test your code regularly as you write it. Use unit tests to verify that each function behaves as expected.
- Consider edge cases and handle potential errors using exceptions.

Use Libraries and Frameworks:

- Take advantage of Python's extensive libraries and frameworks (e.g., NumPy for numerical computing, Pandas for data manipulation) to simplify your tasks and improve efficiency.

Optimize Performance:

- Be mindful of performance issues, especially with larger datasets or complex algorithms. Analyze and optimize your code where necessary.
- Use built-in functions and libraries that are optimized for performance.

Documentation:

- Maintain clear documentation for your projects. This should include an overview of the project, instructions for running the code, and explanations of the main functions and classes.
- Use tools like Sphinx or MkDocs to generate documentation from your docstrings.

Stay Updated:

- Keep up with the latest developments in Python and related libraries. Follow online resources, forums, or newsletters.

Ask Questions:

- If you encounter difficulties or have questions, don't hesitate to ask for help from peers or instructors. Participate in discussions to enhance understanding.

Practice Regularly:

- The more you code, the better you become. Work on personal projects, contribute to open-source projects, or solve coding challenges on platforms like LeetCode or HackerRank.

Collaborate and Share:

- Work with others on projects to learn different approaches and techniques. Share your code on platforms like GitHub to get feedback and contribute to the community.

Follow Ethical Guidelines:

- Respect copyright laws and give proper credit for any external code, libraries, or resources you use. Avoid plagiarism in your work.

INDEX

Exp No.	Expt. Name	Page Number
1	BASICS OF PYTHON PROGRAMMING	10
2	DATA TYPES IN PYTHON PROGRAMMING	14
3	DEMONSTRATE DIFFERENT ARITHMETIC OPERATIONS ON NUMBERS	17
4	STRING OPERATIONS	19
5	TIME AND DATES	21
6	USING NUMPY LIBRARY	23
7	PROGRAM TO CONSTRUCT PATTERNS OF STARS (*) USING A NESTED FOR LOOP	25
8	PROGRAM TO PRINT PRIME NUMBERS LESS THAN N USING A FOR LOOP	28
9	PROGRAM TO FIND THE FACTORIAL OF A NUMBER USING RECURSION	31
10	RECURSIVE FUNCTION TO ADD TWO POSITIVE NUMBERS	34
11	PROGRAM TO MULTIPLY TWO POSITIVE NUMBERS USING RECURSION	37
12	PROGRAM TO FIND THE GREATEST COMMON DIVISOR (GCD) OF TWO POSITIVE NUMBERS USING RECURSION	39
13	RECURSIVE FUNCTION TO MULTIPLY TWO POSITIVE NUMBERS	42
14	RECURSIVE FUNCTION TO THE GREATEST COMMON DIVISOR OF TWO POSITIVE NUMBERS	45

15	RIGHT TRIANGLE IDENTIFICATION USING FUNCTION	48
16	DEFINE A MODULE TO FIND FIBONACCI NUMBERS AND IMPORT THE MODULE TO ANOTHER PROGRAM	51
17	DEFINE A MODULE AND IMPORT A SPECIFIC FUNCTION IN THAT MODULE TO ANOTHER PROGRAM.	54
18	VALIDATION OF MOBILE NUMBER USING FUNCTION	58

EXP NO: 1**SIMPLE DESKTOP CALCULATOR USING PYTHON****AIM:**

To design and implement a simple desktop calculator in Python that performs the five basic arithmetic operations — addition, subtraction, multiplication, and division — based on user choice.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to use conditional statements (if-elif-else) to control program flow based on user input.
- Implement a menu-driven program using loops (while).
- Handle basic input/output operations in Python using input() and print().
- Perform type conversion (float()) for numeric operations.
- Handle a simple runtime error condition (division by zero).

DESCRIPTION:

A menu-driven calculator is a program that repeatedly displays a set of operation choices to the user and performs the selected operation until the user chooses to exit. This program demonstrates decision-making constructs and basic arithmetic operations in Python, forming a foundation for understanding control flow and function-based problem solving.

ALGORITHM:

1. Start
2. Display a menu of available operations (Addition, Subtraction, Multiplication, Division, Exit).
3. Initialize a variable choice to store the user's input.
4. Prompt the user to enter their choice (1–5).
5. Read the user's input and store it in the choice variable.
6. If choice is '5', terminate the program.
7. Otherwise, prompt the user to enter the first number (num1) and the second number (num2).
8. Use if-elif conditions to determine the operation based on choice:
 - If choice == '1': result = num1 + num2
 - If choice == '2': result = num1 - num2

- If choice == '3': result = num1 * num2
 - If choice == '4':
 - Check if num2 is 0. If so, display a "division by zero" error.
 - Otherwise, result = num1 / num2
9. Display the calculated result.
 10. Repeat from step 2 until the user chooses to exit.
 11. Stop

PROCEDURE:

1. Open a Python IDE or text editor (e.g., IDLE, VS Code, Jupyter Notebook).
2. Create a new file and save it with a .py extension, e.g., calculator.py.
3. Type the source code given below.
4. Save the file.
5. Run the program using the command python calculator.py (or run within the IDE).
6. Enter the choice of operation when prompted, followed by the two numbers.
7. Observe the result printed on the screen.
8. Repeat with different operations and inputs, including 0 as the second number for division, to test error handling.
9. Enter 5 to exit the program.

PROGRAM:

```
def calculator():  
    while True:  
        print("Choose an operation:")  
        print("1. Addition")  
        print("2. Subtraction")  
        print("3. Multiplication")  
        print("4. Division")  
        print("5. Exit")  
        choice = input("Enter your choice (1-5): ")  
        if choice == '5':  
            break  
        num1 = float(input("Enter the first number: "))  
        num2 = float(input("Enter the second number: "))  
        if choice == '1':
```

```
    result = num1 + num2
    print("Result:", result)
elif choice == '2':
    result = num1 - num2
    print("Result:", result)
elif choice == '3':
    result = num1 * num2
    print("Result:", result)
elif choice == '4':
    if num2 == 0:
        print("Error: Division by zero is not allowed.")
    else:
        result = num1 / num2
        print("Result:", result)
else:
    print("Invalid choice. Please enter a number between 1 and 5.")
```

calculator()

EXPECTED OUTPUT:

Choose an operation:

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit

Enter your choice (1-5): 1

Enter the first number: 6

Enter the second number: 2

Result: 8.0

Choose an operation:

1. Addition
2. Subtraction
3. Multiplication

4. Division

5. Exit

Enter your choice (1-5): 4

Enter the first number: 6

Enter the second number: 2

Result: 3.0

Choose an operation:

1. Addition

2. Subtraction

3. Multiplication

4. Division

5. Exit

Enter your choice (1-5): 5

MODEL VIVA QUESTIONS:

1. What is the difference between input() and int()/float() in Python?
2. Why is float() used instead of int() for the numbers in this program?
3. What will happen if the user enters a non-numeric value for num1 or num2? How could you handle this using exceptions?
4. Explain the purpose of the while True: loop in this program.
5. How does the break statement work here, and what would happen without it?

RESULT:

The Python program to design a simple desktop calculator performing the five basic arithmetic operations (addition, subtraction, multiplication, and division) using a menu-driven approach was executed successfully, and the output was verified for all operations, including the division-by-zero error condition.

EXP NO: 2**CREATE, CONCATENATE, AND PRINT A STRING AND ACCESS A SUBSTRING****AIM:**

To write a Python program to create, concatenate, and print a string, and to access a sub-string from a given string.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how strings are created and stored in Python.
- Perform string concatenation using the + operator.
- Understand string indexing and slicing to extract a substring.
- Use print() to display string data along with descriptive labels.

DESCRIPTION:

Strings are sequences of characters in Python and are one of the most commonly used data types. This experiment demonstrates how two strings can be combined (concatenated) into one, and how a specific portion (substring) of a string can be extracted using Python's slicing notation string[start:end].

ALGORITHM:

1. Start
2. Create a string:
 - Assign the value "Hello" to a variable named str1.
3. Concatenate strings:
 - Create a new string str2 by combining the contents of str1 with the string "World!".
4. Print concatenated string:
 - Display the value of str2 on the console.
5. Access substring:
 - Extract a portion of str2 starting from index 6 (inclusive) to index 11 (exclusive) and store it in substring.
6. Print the substring.

7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as string_ops.py.
3. Type the source code given below.
4. Run the program using `python string_ops.py`.
5. Observe the concatenated string and the extracted substring in the output.
6. Try changing the slice indices to observe how different substrings are extracted.

PROGRAM:

```
# Create a string
str1 = "Hello"

# Concatenate strings
str2 = str1 + "World!"

# Print the concatenated string
print("Concatenated String:", str2)

# Access substring
substring = str2[6:11]

# Print the substring
print("Substring:", substring)
```

EXPECTED OUTPUT:

```
Concatenated String: HelloWorld!
Substring: orld!
```

MODEL VIVA QUESTIONS:

1. What is string concatenation, and which operator is used for it in Python?
2. What is the index of the first character in a Python string?
3. Explain how slicing works in `str2[6:11]` — why does it return "orld!" and not include index 11?
4. Are strings mutable or immutable in Python? What is the implication of this?

5. What is the difference between `str1 + str2` and `str1 + " " + str2`?

RESULT:

The Python program to create, concatenate, and print a string, and to access a substring from a given string, was executed successfully, and the output was verified.

EXP NO: 3**FAMILIARIZE TIME AND DATE IN VARIOUS FORMATS****AIM:**

To write a Python script to print the current date and time in a specified format (e.g., "Tue Aug 06 15:18:37 2024").

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to work with Python's built-in datetime module.
- Retrieve the current system date and time.
- Format date and time values using format codes (strftime).

DESCRIPTION:

Python's datetime module provides classes for manipulating dates and times. The strftime() method allows date and time objects to be converted into readable, custom-formatted strings using format specifiers such as %a (abbreviated weekday), %B (full month name), %H:%M:%S (time), and %Y (year).

ALGORITHM:

1. Start
2. Import the datetime module.
3. Get the current date and time using datetime.datetime.now().
4. Format the date and time using the specified format string.
5. Print the formatted date and time.
6. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as date_time.py.
3. Type the source code given below.
4. Run the program using python date_time.py.
5. Observe the current date and time printed in the specified format.
6. Try modifying the format string to display the date/time in other formats.

PROGRAM:

```
import datetime
# Get the current date and time
now = datetime.datetime.now()
# Format the date as required
formatted_date = now.strftime("%a %B %d %H:%M:%S %Z %Y")
# Print the formatted date
print(formatted_date)
```

EXPECTED OUTPUT:

Tue August 06 15:18:37 2024

MODEL VIVA QUESTIONS:

1. Which Python module is used to work with dates and times?
2. What does the now() method of datetime.datetime return?
3. What is the purpose of the strftime() method?
4. What do the format codes %a, %B, %H, %M, %S, and %Y represent?
5. What is the difference between %Z and %z in format strings, and why might %Z show blank on some systems?

RESULT:

The Python script to print the current date and time in the specified format was executed successfully, and the output was verified.

EXP NO: 4**CREATE, APPEND, AND REMOVE LISTS IN PYTHON USING NUMPY****AIM:**

To write a Python program to create, append, and remove elements from a list.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to create an empty list in Python.
- Use the append() method to add elements to a list.
- Use the remove() method to delete a specific element from a list.
- Understand that lists in Python are mutable and ordered.

DESCRIPTION:

Lists are one of Python's most versatile built-in data structures, used to store an ordered, changeable collection of items. This experiment demonstrates basic list operations — creating an empty list, adding elements to it dynamically using append(), and removing a specific element using remove().

ALGORITHM:

1. Start
2. Initialize an empty list named my_list.
3. Add elements 10, 20, and 30 to my_list using the append() method.
4. Print the contents of my_list.
5. Remove the element 20 from my_list using the remove() method.
6. Print the contents of my_list again.
7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as list_ops.py.
3. Type the source code given below.
4. Run the program using python list_ops.py.
5. Observe the list contents before and after removing an element.
6. Try appending and removing other elements to understand list behavior further.

PROGRAM:

```
# Create an empty list
my_list = []
# Append elements to the list
my_list.append(10)
my_list.append(20)
my_list.append(30)
print("List after appending elements:", my_list)
# Remove elements from the list
my_list.remove(20)
print("List after removing elements:", my_list)
```

EXPECTED OUTPUT:

List after appending elements: [10, 20, 30]

List after removing elements: [10, 30]

MODEL VIVA QUESTIONS:

1. What is a list in Python, and how is it different from a tuple?
2. What is the purpose of the append() method?
3. What happens if you try to remove() an element that does not exist in the list?
4. What is the difference between remove() and pop()?
5. How would you insert an element at a specific position in a list?

RESULT:

The Python program to create, append, and remove elements from a list was executed successfully, and the output was verified.

EXP NO: 5**PROGRAM TO FIND THE LARGEST OF THREE NUMBERS****AIM:**

To write a Python program to find the largest of three numbers.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to take multiple numeric inputs from the user.
- Use Python's built-in max() function to compare values.
- Perform type conversion of user input to float.

DESCRIPTION:

Finding the largest among a set of numbers is a fundamental problem-solving task. Instead of using multiple if-else comparisons, Python provides a built-in max() function that can directly compare multiple values and return the largest one, simplifying the logic considerably.

ALGORITHM:

1. Start
2. Initialize three numbers by taking input from the user and storing them as num1, num2, and num3.
3. Use the max() function to find the greatest number among num1, num2, and num3.
4. Store the result in a variable largest.
5. Print the largest number.
6. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as largest_of_three.py.
3. Type the source code given below.
4. Run the program using python largest_of_three.py.
5. Enter three numbers when prompted.
6. Observe the largest number displayed in the output.
7. Repeat with different sets of numbers, including negative numbers and duplicates, to test the program.

PROGRAM:

```
# Prompt the user to enter three numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))
# Find the largest number
largest = max(num1, num2, num3)
# Print the largest number
print("The largest number is:", largest)
```

EXPECTED OUTPUT:

```
Enter the first number: 12
Enter the second number: 22
Enter the third number: 43
The largest number is: 43.0
```

MODEL VIVA QUESTIONS:

1. What does the max() function do in Python, and what data types can it accept?
2. Why is float() used to convert the input values?
3. How would you find the largest number without using the max() function (using if-else)?
4. How would you modify the program to also find the smallest number?
5. What happens if two or more numbers entered are equal?

RESULT:

The Python program to find the largest of three numbers was executed successfully, and the output was verified.

EXP NO: 6

CONVERT TEMPERATURE VALUES BETWEEN CELSIUS AND FAHRENHEIT

AIM:

To write a Python program to convert temperature values back and forth between Celsius (C) and Fahrenheit (F), using the formula: $C/5 = (F-32)/9$.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to apply a mathematical formula in a Python program.
- Perform arithmetic operations involving multiplication, division, and addition.
- Use formatted string output (% operator formatting) to display results with a specific number of decimal places.

DESCRIPTION:

Temperature conversion between Celsius and Fahrenheit is a classic beginner programming problem that demonstrates the direct translation of a mathematical formula into code. The relationship between Celsius and Fahrenheit is given by: $F = (C \times 1.8) + 32$, derived from the standard formula $C/5 = (F-32)/9$.

ALGORITHM:

1. Start
2. Assign a numerical value to the variable celsius.
3. Calculate the equivalent temperature in Fahrenheit using the formula: $fahrenheit = (celsius * 1.8) + 32$.
4. Print the calculated Fahrenheit value along with the original Celsius value in a formatted string.
5. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as temp_convert.py.
3. Type the source code given below.
4. Run the program using `python temp_convert.py`.
5. Observe the converted Fahrenheit value in the output.

6. Modify the celsius value to test the conversion for different temperatures.
7. (Optional extension) Modify the program to also accept user input and convert Fahrenheit to Celsius.

PROGRAM:

```
# Python Program to convert temperature in celsius to fahrenheit
# change this value for a different result
celsius = 34

# calculate fahrenheit
fahrenheit = (celsius * 1.8) + 32

print('%0.1f degree Celsius is equal to %0.1f degree Fahrenheit' % (celsius, fahrenheit))
```

EXPECTED OUTPUT:

34.0 degree Celsius is equal to 93.2 degree Fahrenheit

MODEL VIVA QUESTIONS:

1. What is the formula used to convert Celsius to Fahrenheit?
2. How would you convert Fahrenheit back to Celsius? Write the formula.
3. What does %0.1f mean in the print statement's format specifier?
4. What is the difference between the % formatting style and the .format() method or f-strings in Python?
5. Why is 1.8 used instead of 9/5 in the formula? Are they equivalent?

RESULT:

The Python program to convert temperature values between Celsius and Fahrenheit was executed successfully, and the output was verified.

EXP NO: 7

PROGRAM TO CONSTRUCT PATTERNS OF STARS (*) USING A NESTED FOR LOOP

AIM:

To write a Python program to construct a diamond-shaped pattern of stars (*) using a nested for loop.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the working of nested loops in Python.
- Control loop iteration ranges to generate increasing and decreasing patterns.
- Use the end parameter of the print() function to control line breaks and spacing.

DESCRIPTION:

Pattern printing problems are widely used to build a strong understanding of loop control and nested iteration logic. This program constructs a diamond pattern of stars by first printing rows with an increasing number of stars, then printing rows with a decreasing number of stars, using two separate nested loops.

ALGORITHM:

1. Start
2. Initialize a variable n to determine the number of rows (here, n = 5).
3. Create an outer loop to iterate through the rows:
 - The loop will iterate from 1 to n for the increasing part of the pattern.
4. Create an inner loop to print stars in each row of the increasing part, iterating from 1 up to the current row number.
5. Print a star followed by a space within the inner loop, without moving to a new line.
6. Print a newline after each row.
7. Create a second outer loop to iterate from n down to 1 for the decreasing part of the pattern.
8. Create an inner loop to print stars in each row of the decreasing part.
9. Print a newline after each row.
10. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as star_pattern.py.
3. Type the source code given below.
4. Run the program using python star_pattern.py.
5. Observe the diamond-shaped star pattern in the output.
6. Change the value of n to test the pattern for different sizes.

PROGRAM:

```
n = 5

# Increasing part of the pattern
for i in range(n):
    for j in range(i):
        print('* ', end='')
    print("")

# Decreasing part of the pattern
for i in range(n, 0, -1):
    for j in range(i):
        print('* ', end='')
    print("")
```

EXPECTED OUTPUT:

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

MODEL VIVA QUESTIONS:

1. What is a nested loop, and why is it needed to print this pattern?
2. What does the `end=""` parameter do in the `print()` function?
3. Explain the difference between `range(n)` and `range(n, 0, -1)`.
4. Why does the first `for i in range(n)` loop only produce up to 4 stars in the last row instead of 5?
5. How would you modify the code to correctly print 5 stars in the middle row (i.e., fix the off-by-one issue)?

RESULT:

The Python program to construct a diamond pattern of stars using nested for loops was executed successfully, and the output was verified.

EXP NO: 8**A PROGRAM THAT PRINTS PRIME NUMBERS LESS THAN N****AIM:**

To write a Python program that prints all prime numbers less than a given number N.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the definition and logic behind identifying prime numbers.
- Implement a helper function (isPrime) that can be reused within a driver function.
- Use nested loops and conditional checks to solve a mathematical problem.
- Understand the role of the `__name__ == "__main__"` construct in Python.

DESCRIPTION:

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. This program uses a function `isPrime()` to check whether a given number is prime by testing divisibility from 2 up to $n-1$, and a driver function `printPrime()` that iterates through numbers up to N, printing each one that is prime.

ALGORITHM:

1. Start
2. Define a function `isPrime(n)`:
 - If the number is less than or equal to 1, return False (not prime).
 - Iterate from 2 to $n-1$:
 - If the number is divisible by the current iterator, return False (not prime).
 - If the loop completes without finding a divisor, return True (prime).
3. Define a function `printPrime(n)`:
 - Iterate from 2 to n (inclusive).
 - If `isPrime(i)` returns True, print the number.
4. In the driver code, set $n = 20$ and call `printPrime(n)`.
5. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as prime_numbers.py.
3. Type the source code given below.
4. Run the program using python prime_numbers.py.
5. Observe the list of prime numbers less than N printed in the output.
6. Change the value of n to test the program for different ranges.

PROGRAM:

```
def isPrime(n):  
    # Corner case  
    if n <= 1:  
        return False  
    # check from 2 to n-1  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True  
  
# Function to print primes  
def printPrime(n):  
    for i in range(2, n + 1):  
        if isPrime(i):  
            print(i, end=" ")  
  
# Driver code  
if __name__ == "__main__":  
    n = 20  
    # function calling  
    printPrime(n)
```

EXPECTED OUTPUT:

2 3 5 7 11 13 17 19

MODEL VIVA QUESTIONS:

1. What is the definition of a prime number?
2. Why does isPrime() return False immediately for numbers less than or equal to 1?
3. What is the time complexity of the isPrime() function as implemented here?
4. How can the isPrime() function be optimized using the square root of n instead of checking up to n-1?
5. What does if __name__ == "__main__": mean, and why is it used?

RESULT:

The Python program to print all prime numbers less than a given number N was executed successfully, and the output was verified.

EXP NO: 9**PROGRAM TO FIND THE FACTORIAL OF A NUMBER USING RECURSION****AIM:**

To write a Python program to find the factorial of a number using recursion.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the concept of recursion and how a function calls itself.
- Identify the base case and recursive case in a recursive function.
- Understand how the call stack builds up and unwinds during recursive execution.
- Handle edge cases such as zero and negative inputs.

DESCRIPTION:

Recursion is a programming technique where a function calls itself to solve a smaller instance of the same problem, until it reaches a base case that can be solved directly. The factorial of a number n (denoted $n!$) is the product of all positive integers from 1 to n , and is a classic example used to illustrate recursion, since $n! = n \times (n-1)!$.

ALGORITHM:

1. Start
2. Define a recursive function `recur_factorial(n)`:
 - If n is equal to 1, return 1 (base case).
 - If n is greater than 1, return n multiplied by the result of calling `recur_factorial(n-1)` (recursive case).
3. Get the number `num` whose factorial is to be found.
4. Check if `num` is negative:
 - If negative, print an error message.
5. Check if `num` is zero:
 - If zero, print that the factorial is 1.
6. If `num` is positive:
 - Calculate the factorial using `recur_factorial(num)` and print the result.
7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as factorial_recursion.py.
3. Type the source code given below.
4. Run the program using python factorial_recursion.py.
5. Observe the factorial value printed in the output.
6. Change the value of num to test the program for zero, positive, and negative inputs.

PROGRAM:

```
def recur_factorial(n):  
    if n == 1:  
        return n  
    else:  
        return n * recur_factorial(n - 1)  
  
num = 5  
  
# check if the number is negative  
if num < 0:  
    print("Sorry, factorial does not exist for negative numbers")  
elif num == 0:  
    print("The factorial of 0 is 1")  
else:  
    print("The factorial of", num, "is", recur_factorial(num))
```

EXPECTED OUTPUT:

The factorial of 5 is 120

MODEL VIVA QUESTIONS:

1. What is recursion? Explain with reference to this program.
2. What is the base case in recur_factorial(), and why is it necessary?
3. What would happen if the base case were removed from the recursive function?
4. Trace the call stack for recur_factorial(5) — show each recursive call and its return value.

5. What is the difference between recursion and iteration? Which is more memory-efficient for factorial calculation?

RESULT:

The Python program to find the factorial of a number using recursion was executed successfully, and the output was verified.

EXP NO: 10**RECURSIVE FUNCTION TO ADD TWO POSITIVE NUMBERS****AIM:**

To write a Python program to add two positive numbers using a recursive function.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how to define and call a function in Python.
- Take multiple numeric inputs from the user and pass them as function arguments.
- Understand the concept of recursion and how addition can be expressed recursively.
- Use formatted string output (f-strings) to display results.

DESCRIPTION:

Addition of two numbers can be implemented either directly or recursively by repeatedly incrementing/decrementing one number while incrementing/decrementing the other, until one of them reaches zero. This experiment introduces the use of functions with parameters and return values, laying the foundation for recursive problem solving.

ALGORITHM:

1. Start
2. Get two numbers as input from the user and store them in number1 and number2.
3. Call the add_numbers function with number1 and number2 as arguments.
4. Calculate the sum of num1 and num2.
5. Return the calculated sum.
6. Store the returned sum in the result variable.
7. Print the calculated sum.
8. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as add_recursive.py.

3. Type the source code given below.
4. Run the program using python add_recursive.py.
5. Enter two numbers when prompted.
6. Observe the sum displayed in the output.
7. Repeat with different pairs of numbers, including decimals.

PROGRAM:

```
def add_numbers(num1, num2):  
    sum_result = num1 + num2  
    return sum_result  
  
# Input from the user  
number1 = float(input("Enter the first number: "))  
number2 = float(input("Enter the second number: "))  
  
# Call the add_numbers function and display the result  
result = add_numbers(number1, number2)  
print(f"The sum of {number1} and {number2} is: {result}")
```

EXPECTED OUTPUT:

```
Enter the first number: 25  
Enter the second number: 30  
The sum of 25.0 and 30.0 is: 55.0
```

MODEL VIVA QUESTIONS:

1. Is the add_numbers function in this program truly recursive? Justify your answer.
2. How would you rewrite this function to perform addition recursively (e.g., using increment/decrement)?
3. What is the base case you would use for a recursive addition function?
4. What is the difference between a function call and a recursive function call?
5. Why is float() used to convert user input, and what would happen if int() were used with decimal input?

RESULT:

The Python program to add two positive numbers using a function was executed successfully, and the output was verified.

EXP NO: 11**RECURSIVE FUNCTION TO MULTIPLY TWO POSITIVE NUMBERS****AIM:**

To write a Python program to multiply two positive numbers using a recursive function.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand how multiplication can be expressed as repeated addition using recursion.
- Identify the base case and recursive case for a multiplication recursive function.
- Trace the execution of a recursive function through the call stack.
- Understand how recursion reduces a problem to a smaller sub-problem at each step.

DESCRIPTION:

Multiplication of two positive numbers x and y can be defined recursively as repeated addition: $x \times y = x + x \times (y-1)$, with the base case being $x \times 0 = 0$. This experiment demonstrates how a seemingly non-recursive arithmetic operation (multiplication) can be implemented using pure recursion.

ALGORITHM:

1. Start
2. Define a recursive function `multiply_recursive(x, y)` that takes two numbers, x and y , and returns their product:
 - If the second number, y , is 0, the product is 0. This is the base case that stops the recursion; return 0.
 - Otherwise, recursively calculate the product of x and $y - 1$, and add x to the result of the recursive call: return $x + \text{multiply_recursive}(x, y - 1)$.
3. The final result of the multiplication is returned, either from the base case or the recursive case.
4. Call the function with the two numbers and print the result.
5. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as `multiply_recursive.py`.
3. Type the source code given below.
4. Run the program using `python multiply_recursive.py`.

5. Observe the product printed in the output.
6. Change the values of num1 and num2 to test the program with different inputs.

PROGRAM:

```
def multiply_recursive(x, y):  
    if y == 0:  
        return 0  
    else:  
        return x + multiply_recursive(x, y - 1)  
  
num1 = 4  
num2 = 8  
  
result = multiply_recursive(num1, num2)  
  
print(f"The product of {num1} and {num2} is: {result}")
```

EXPECTED OUTPUT:

The product of 4 and 8 is: 32

MODEL VIVA QUESTIONS:

1. What is the base case in multiply_recursive(), and why is y used for the base case instead of x?
2. Explain how multiplication is reduced to repeated addition in this recursive function.
3. Trace the recursive calls for multiply_recursive(4, 8) step by step, showing the return values.
4. What would happen if y were negative? Would the function terminate?
5. How would you modify the function to swap the roles of x and y for efficiency (i.e., recurse on the smaller number)?

RESULT:

The Python program to multiply two positive numbers using a recursive function was executed successfully, and the output was verified.

EXP NO: 12

RECURSIVE FUNCTION TO FIND THE GREATEST COMMON DIVISOR OF TWO POSITIVE NUMBERS

AIM:

To write a Python program to find the Greatest Common Divisor (GCD) of two positive numbers using a recursive function.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the Euclidean algorithm for computing the GCD of two numbers.
- Implement recursion using the modulus operator.
- Identify the base case for GCD recursion.
- Understand how recursive calls reduce the problem size at each step until termination.

DESCRIPTION:

The Greatest Common Divisor (GCD) of two numbers is the largest positive integer that divides both numbers without leaving a remainder. This program uses the Euclidean algorithm, which states that $\text{GCD}(\text{num1}, \text{num2}) = \text{GCD}(\text{num2}, \text{num1} \% \text{num2})$, and the recursion terminates when the second number becomes 0, at which point the first number is the GCD.

ALGORITHM:

1. Start
2. Input two positive integers, num1 and num2.
3. Define a recursive function `greatest_common_divisor(num1, num2)`:
 - If num2 is 0, return num1 (base case).
 - If num2 is not 0:
 - Calculate the remainder of num1 divided by num2 and store it in a variable remainder.
 - Recursively call the function with num2 as the first number and remainder as the second number.
 - Return the result of the recursive call.

4. Call the function with num1 and num2, and print the result.
5. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as gcd_recursive.py.
3. Type the source code given below.
4. Run the program using python gcd_recursive.py.
5. Observe the GCD value printed in the output.
6. Change the values of num1 and num2 to test the program with different inputs.

PROGRAM:

```
def greatest_common_divisor(num1, num2):  
    # Base case: If num2 is 0, the GCD is num1  
    if num2 == 0:  
        return num1  
    # Recursive case: GCD of num1 and num2 is the same as GCD of num2  
    # and the remainder of num1 divided by num2  
    return greatest_common_divisor(num2, num1 % num2)  
  
# Example usage:  
num1 = 24  
num2 = 18  
result = greatest_common_divisor(num1, num2)  
print("The GCD of", num1, "and", num2, "is:", result)
```

EXPECTED OUTPUT:

The GCD of 24 and 18 is: 6

MODEL VIVA QUESTIONS:

1. What is the Euclidean algorithm, and how is it used to compute the GCD?
2. What is the base case in greatest_common_divisor(), and why does it use num2 == 0?
3. Trace the recursive calls for greatest_common_divisor(24, 18) step by step.
4. Why does the Euclidean algorithm use the modulus operator (%) instead of subtraction?
5. What is the time complexity of the Euclidean algorithm for finding GCD?

RESULT:

The Python program to find the greatest common divisor of two positive numbers using a recursive function was executed successfully, and the output was verified.

EXP NO: 13**PROGRAM TO CHECK WHETHER THREE SIDES FORM A RIGHT TRIANGLE****AIM:**

To write a Python program that accepts the lengths of three sides of a triangle as inputs and determines whether the triangle is a right triangle, using functions.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Apply the Pythagorean Theorem programmatically.
- Use the sort() method to arrange values in a defined order.
- Implement exception handling using try-except for invalid inputs.
- Structure a program using a main() function and the if `__name__ == "__main__":` construct.

DESCRIPTION:

The Pythagorean Theorem states that in a right triangle, the square of the length of the hypotenuse (the longest side) is equal to the sum of the squares of the lengths of the other two sides. This program sorts the three input side lengths to correctly identify the hypotenuse, then checks whether the Pythagorean relationship holds.

ALGORITHM

1. Start
2. Get the lengths of three sides of a triangle from the user and store them in variables side1, side2, and side3.
3. Sort the three side lengths in ascending order. This ensures that the longest side is always at the end.
4. Calculate the square of the longest side (hypotenuse).
5. Calculate the sum of the squares of the other two sides.
6. Compare the results of the two calculations.
7. If the square of the longest side is equal to the sum of the squares of the other two sides, print "The triangle is a right triangle."
8. Otherwise, print "The triangle is not a right triangle."
9. Handle invalid (non-numeric) input using exception handling.
10. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as right_triangle.py.
3. Type the source code given below.
4. Run the program using python right_triangle.py.
5. Enter the three side lengths when prompted.
6. Observe whether the program correctly identifies a right triangle.
7. Test with known right-triangle values (e.g., 3, 4, 5) and non-right-triangle values (e.g., 5, 8, 2).
8. Test with invalid (non-numeric) input to verify the exception handling.

PROGRAM:

```
def is_right_triangle(side1, side2, side3):
    sides = [side1, side2, side3]
    sides.sort() # Ensure the longest side is the last element
    return sides[2]**2 == sides[0]**2 + sides[1]**2

def main():
    try:
        side1 = float(input("Enter the length of the first side: "))
        side2 = float(input("Enter the length of the second side: "))
        side3 = float(input("Enter the length of the third side: "))

        if is_right_triangle(side1, side2, side3):
            print("The triangle is a right triangle.")
        else:
            print("The triangle is not a right triangle.")
    except ValueError:
        print("Invalid input. Please enter numerical values for the side lengths.")

if __name__ == "__main__":
    main()
```

EXPECTED OUTPUT:

Enter the length of the first side: 3

Enter the length of the second side: 4

Enter the length of the third side: 5

The triangle is a right triangle.

Enter the length of the first side: 5

Enter the length of the second side: 8

Enter the length of the third side: 2

The triangle is not a right triangle.

MODEL VIVA QUESTIONS:

1. State the Pythagorean Theorem and explain how it is applied in this program.
2. Why is the sides list sorted before checking the right-triangle condition?
3. What would go wrong if the sides were not sorted before the check?
4. What is the purpose of the try-except block in this program? What exception is being caught?
5. What would happen if a negative number were entered as a side length? Does the program handle this correctly?

RESULT:

The Python program to check whether the given three sides of a triangle form a right triangle was executed successfully, and the output was verified.

EXP NO: 14**PROGRAM TO DEFINE A MODULE TO FIND FIBONACCI NUMBERS AND
IMPORT IT****AIM:**

To write a Python program to define a module to find Fibonacci numbers and import the module into another program.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the concept of a module in Python and how to create a user-defined module.
- Use the import statement to access functions from another Python file.
- Implement recursion to compute Fibonacci numbers.
- Understand the relationship between module files and the programs that use them.

DESCRIPTION:

A module in Python is simply a .py file containing reusable functions, classes, or variables that can be imported into other programs. This experiment demonstrates modular programming by defining a fibonacci() function in a separate file (fibonacci.py) that recursively computes the nth Fibonacci number, and then importing and using this module from a second program (main.py).

ALGORITHM:

1. Start
2. In a module file (fibonacci.py), define a function fibonacci(n):
 - Takes an integer n as input.
 - If n is 0 or 1, return the corresponding Fibonacci number (0 or 1).
 - Otherwise, recursively call itself with n-1 and n-2 and return their sum.
3. In a second file (main.py), import the fibonacci module.
4. Prompt the user to enter the value of n.
5. Convert the input to an integer.
6. Calculate the nth Fibonacci number by calling fibonacci.fibonacci(n).
7. Print the calculated Fibonacci number.
8. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file named fibonacci.py and type the module code (the fibonacci() function) into it. Save it.
3. Create a second file named main.py in the **same folder** as fibonacci.py.
4. Type the main program code (which imports and uses the fibonacci module) into main.py. Save it.
5. Run the program using python main.py.
6. Enter the value of n when prompted.
7. Observe the nth Fibonacci number displayed in the output.
8. Test with different values of n to verify correctness.

PROGRAM:

fibonacci.py (module file)

```
def fibonacci(n):  
    """Calculates the nth Fibonacci number."""  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)
```

main.py (imports and uses the module)

```
import fibonacci  
  
n = int(input("Enter the value of n: "))  
fib_number = fibonacci.fibonacci(n)  
print("The nth Fibonacci number is:", fib_number)
```

EXPECTED OUTPUT:

Enter the value of n: 7
The nth Fibonacci number is: 13

MODEL VIVA QUESTIONS

1. What is a module in Python? How is it different from a package?
2. Why must fibonacci.py and main.py be saved in the same directory for import fibonacci to work?
3. What is the difference between import fibonacci and from fibonacci import fibonacci?
4. Explain the base cases in the fibonacci() function. Why are there two base cases (for n=0 and n=1)?
5. Trace the recursive calls made to compute fibonacci(7), up to the first few levels.

RESULT:

The Python program to define a module to find Fibonacci numbers and import it into another program was executed successfully, and the output was verified.

EXP NO: 15**PROGRAM TO CHECK WHETHER A GIVEN NUMBER IS A VALID MOBILE
NUMBER****AIM:**

To write a Python program to check whether the given number is a valid mobile number or not using functions.

Rules:

1. Every number should contain exactly 10 digits.
2. The first digit should be 7, 8, or 9.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand the concept of regular expressions (regex) in Python.
- Use the re module to validate string patterns.
- Understand pattern-matching functions like re.fullmatch().
- Apply pattern-based validation to real-world input scenarios.

DESCRIPTION:

Regular expressions provide a powerful and concise way to match and validate patterns within strings. In this experiment, a regular expression is used to check whether an input string represents a valid 10-digit mobile number starting with 7, 8, or 9, using Python's built-in re module.

ALGORITHM:

1. Start
2. Import the re module: this module is necessary for regular expression operations.
3. Get user input: prompt the user to enter a mobile number and store it in the variable s.
4. Define the regular expression pattern: '[7-9][0-9]{9}' specifies that the number must start with a digit between 7 and 9 and have a total length of 10 digits.
5. Check for a full match: use re.fullmatch() to determine if the entire input string matches the pattern.
6. Print the result: if there's a full match, print "Valid"; otherwise, print "Invalid".
7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as mobile_validate.py.
3. Type the source code given below.
4. Run the program using python mobile_validate.py.
5. Enter a mobile number when prompted.
6. Observe whether the program correctly reports it as valid or invalid.
7. Test with numbers of different lengths, numbers starting with digits other than 7/8/9, and numbers containing non-digit characters.

PROGRAM:

```
import re

s = input('Enter Number :')
m = re.fullmatch('[7-9][0-9]{9}', s)

if m != None:
    print(s, 'is valid Mobile number')
else:
    print(s, 'is not valid Mobile number')
```

EXPECTED OUTPUT:

```
Enter Number :7894562583
7894562583 is valid Mobile number
```

```
Enter Number :123568
123568 is not valid Mobile number
```

MODEL VIVA QUESTIONS:

1. What is a regular expression, and what module provides regex support in Python?
2. Explain the pattern '[7-9][0-9]{9}' in detail — what does each part match?
3. What is the difference between re.match(), re.search(), and re.fullmatch()?
4. Why is fullmatch() specifically used here instead of match()?

5. What would happen if the input contained 11 digits instead of 10? Would it be considered valid?

RESULT:

The Python program to check whether a given number is a valid mobile number was executed successfully, and the output was verified.

EXP NO: 16**MERGE TWO LISTS — EVEN NUMBERS FIRST, THEN ODD NUMBERS,
BOTH SORTED****AIM:**

To input two lists from the user and merge them into a third list such that all even numbers occur first, followed by all odd numbers, with both groups arranged in sorted order.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Use list comprehension to filter elements based on a condition.
- Combine two lists using the + operator.
- Use the sorted() function and the extend() method to build a new list.
- Apply the modulus operator to distinguish even and odd numbers.
- Parse user input into a list of integers using map() and split().

DESCRIPTION:

This program demonstrates combining multiple list-processing techniques: filtering (using list comprehension with a condition), sorting, and merging. The task is to take two lists as input, then form a merged list where all even numbers (in ascending sorted order) appear first, followed by all odd numbers (also in ascending sorted order).

ALGORITHM:

1. Start
2. Create an empty list: merged_list is initialized to store the combined elements from both input lists.
3. Sort and append even numbers:
 - Use list comprehension to extract even numbers from both lists (list1 and list2).
 - Sort these even numbers using the sorted() function.
 - Append the sorted even numbers to merged_list.
4. Sort and append odd numbers:
 - Similarly, extract odd numbers from both lists using list comprehension.
 - Sort them and append them to merged_list.

5. Return the merged list.
6. Get the two input lists from the user (space-separated integers).
7. Call the merge function and print the resulting merged list.
8. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as merge_lists.py.
3. Type the source code given below.
4. Run the program using python merge_lists.py.
5. Enter the elements of the first list, separated by spaces, when prompted.
6. Enter the elements of the second list, separated by spaces, when prompted.
7. Observe the merged list (evens sorted, then odds sorted) in the output.
8. Test with lists containing negative numbers, duplicates, and all-even or all-odd lists.

PROGRAM:

```
def merge_lists(list1, list2):  
    """Merges two lists, placing even numbers first in sorted order,  
    followed by odd numbers in sorted order."""  
    merged_list = []  
  
    # Sort even numbers from both lists and append to merged list  
    merged_list.extend(sorted([num for num in list1 + list2 if num % 2 == 0]))  
  
    # Sort odd numbers from both lists and append to merged list  
    merged_list.extend(sorted([num for num in list1 + list2 if num % 2 != 0]))  
  
    return merged_list  
  
# Get input lists from the user  
list1 = list(map(int, input("Enter elements of the first list separated by spaces: ").split()))  
list2 = list(map(int, input("Enter elements of the second list separated by spaces: ").split()))  
  
# Merge the lists and print the result  
merged_list = merge_lists(list1, list2)  
print("Merged list:", merged_list)
```

EXPECTED OUTPUT:

Enter elements of the first list separated by spaces: 5 2 7 6 9

Enter elements of the second list separated by spaces: 1 4 3 8

Merged list: [2, 4, 6, 8, 1, 3, 5, 7, 9]

MODEL VIVA QUESTIONS:

1. What does `list(map(int, input().split()))` do? Break it down step by step.
2. Explain the list comprehension `[num for num in list1 + list2 if num % 2 == 0]`.
3. What is the time complexity of this merging approach, considering the two `sorted()` calls?
4. Why does the program iterate over `list1 + list2` twice (once for evens, once for odds)? How could this be made more efficient with a single pass?
5. How does the modulus operator (%) determine whether a number is even or odd?

RESULT:

The Python program to merge two lists such that even numbers appear first (sorted) followed by odd numbers (sorted) was executed successfully, and the output was verified.

EXP NO: 17**PROGRAM TO PLAY A STICKS GAME****AIM:**

To write a Python program to simulate a sticks game with 16 sticks, where two players take turns picking 1, 2, or 3 sticks, and the player who picks the last stick loses.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Implement game logic involving turn-based player interaction.
- Use a while loop to control game flow until a terminating condition is met.
- Validate user input against multiple constraints.
- Understand how to alternate between two players using conditional logic.
- Apply backtracking/strategy concepts (Nim-like game) in a simple algorithmic form.

DESCRIPTION:

This program simulates a variant of the classic Nim game. Sixteen sticks are placed on the table, and two players alternately remove 1, 2, or 3 sticks from the pile on their turn. Unlike standard Nim, in this "misère" version, the player forced to take the last stick loses the game. The program validates each player's move to ensure it follows the rules before updating the stick count and switching turns.

ALGORITHM:

1. Start
2. Set the initial number of sticks to 16.
3. Set the current player to 1.
4. While there are still sticks remaining:
 - Print the current number of sticks.
 - Prompt the current player to choose how many sticks to pick (1–3).
 - Validate the input to ensure it's within the allowed range and doesn't exceed the remaining sticks.
 - If the input is invalid, ask the player to enter a valid number and continue.
 - Subtract the chosen number of sticks from the total.

- Switch to the other player.
5. The player who picks the last stick loses the game.
 6. Print a message announcing the loser.
 7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as sticks_game.py.
3. Type the source code given below.
4. Run the program using python sticks_game.py.
5. Play the game by entering the number of sticks (1–3) to pick when prompted, alternating between Player 1 and Player 2.
6. Continue until all sticks are exhausted.
7. Observe which player is declared the loser.
8. Test with invalid entries (e.g., 0, 4, or a number exceeding the remaining sticks) to verify input validation.

PROGRAM:

```
def play_sticks():
    """Plays the sticks game where two players take turns picking sticks."""
    sticks = 16
    current_player = 1

    while sticks > 0:
        print(f"\nCurrent sticks: {sticks}")

        # Get the number of sticks to pick
        pick = int(input(f"Player {current_player}, how many sticks do you want to pick (1-3): "))

        # Validate the input
        if pick < 1 or pick > 3 or pick > sticks:
            print("Invalid input. Please enter a number between 1 and 3, and ensure it doesn't exceed the remaining sticks.")
            continue

        # Update the number of sticks
```

```
sticks -= pick

# Switch players
current_player = 2 if current_player == 1 else 1

# Determine the loser
loser = 2 if current_player == 1 else 1
print(f"\nPlayer {loser} loses the game!")

if __name__ == "__main__":
    play_sticks()
```

EXPECTED OUTPUT:

```
Current sticks: 16
Player 1, how many sticks do you want to pick (1-3): 3

Current sticks: 13
Player 2, how many sticks do you want to pick (1-3): 2

Current sticks: 11
Player 1, how many sticks do you want to pick (1-3): 3

Current sticks: 8
Player 2, how many sticks do you want to pick (1-3): 1

Current sticks: 7
Player 1, how many sticks do you want to pick (1-3): 3

Current sticks: 4
Player 2, how many sticks do you want to pick (1-3): 2

Current sticks: 2
Player 1, how many sticks do you want to pick (1-3): 2
```

Player 1 loses the game!

MODEL VIVA QUESTIONS:

1. What is the rule of this sticks game, and how does it differ from standard Nim?
2. Why is the `current_player` value checked/switched *after* subtracting the sticks rather than before?
3. Explain the input validation condition: `pick < 1` or `pick > 3` or `pick > sticks`. Why is each check needed?
4. Why does the `continue` statement not increment `current_player`? What effect does this have on game logic?
5. Is there a winning strategy for this game? How would you determine the optimal number of sticks to pick given a certain remaining count (hint: relates to `sticks % 4`)?

RESULT:

The Python program to simulate the sticks game between two players was executed successfully, and the output was verified.

EXP NO: 18**SIMULATION OF THE MONTY HALL PROBLEM****AIM:**

To write a Python program to simulate the Monty Hall problem and determine whether switching doors after the host reveals a goat increases the player's chances of winning the car.

LEARNING OUTCOMES:

By completing this experiment, students will be able to:

- Understand and apply the concept of the Randomized Approach to problem-solving.
- Use Python's random module to simulate probabilistic scenarios.
- Use Monte Carlo simulation (repeated trials) to empirically validate a probability puzzle.
- Understand set operations in Python (union, difference, pop()) to model constrained random choices.
- Interpret simulation results to draw statistical conclusions.

DESCRIPTION:

The Monty Hall problem is a classic probability puzzle based on a game show scenario: a contestant picks one of three doors, behind one of which is a car and behind the other two are goats. After the choice, the host — who knows what's behind each door — opens one of the remaining doors to reveal a goat, and then offers the contestant the choice to switch to the other unopened door. Although counterintuitive, switching doors gives the contestant a higher probability ($2/3$) of winning the car compared to staying ($1/3$). This program uses a Monte Carlo simulation — running the scenario thousands of times — to empirically demonstrate this result.

(Source: https://en.wikipedia.org/wiki/Monty_Hall_problem)

ALGORITHM:

1. Start
2. Initialize variables: set the number of doors and the number of simulations.
3. Initialize variables to keep track of wins for staying and switching.
4. Repeat the following steps for the specified number of simulations:
 - Place the car behind a random door.

- Player picks a random door.
 - Host reveals a goat behind a different door (neither the player's chosen door nor the car's door).
 - If switching doors, choose the remaining (third) door; otherwise, stay with the original choice.
 - Check if the player's final door matches the car's door — if so, it's a win.
 - Increment the win counter for the appropriate strategy (stay or switch).
5. Calculate win percentages: divide the number of wins for each strategy by the total number of simulations and calculate the percentages.
 6. Display the win counts and percentages for both strategies, along with a conclusion indicating that switching doors is advantageous.
 7. Stop

PROCEDURE:

1. Open a Python IDE or text editor.
2. Create a new file and save it as `monty_hall.py`.
3. Type the source code given below.
4. Run the program using `python monty_hall.py`.
5. Wait for the program to complete 10,000 simulated games for both the "stay" and "switch" strategies.
6. Observe the win counts and win percentages for each strategy in the output.
7. Compare the two percentages to confirm that switching yields a higher win rate (close to 66.7%) compared to staying (close to 33.3%).
8. Re-run the program multiple times to observe how the results converge close to the theoretical probabilities as the number of simulations is large.

PROGRAM:

```
import random

def play_monty_hall(switch_doors=True):
    """Simulates the Monty Hall problem with or without switching doors."""
    # Number of doors
    doors = 3

    # Place the car behind a random door
```

```
car_door = random.randint(1, doors)

# Player picks a random door
chosen_door = random.randint(1, doors)

# Host reveals a goat behind a different door (not the chosen or car door)
revealed_door = (set(range(1, doors + 1)) - {chosen_door, car_door}).pop()

# Player switches or stays
new_door = chosen_door if not switch_doors else (set(range(1, doors + 1)) - {chosen_door,
revealed_door}).pop()

# Check if the player wins
return new_door == car_door

# Simulate the game multiple times with and without switching doors
num_simulations = 10000
wins_stay = 0
wins_switch = 0

for _ in range(num_simulations):
    wins_stay += not play_monty_hall(switch_doors=False)
    wins_switch += play_monty_hall(switch_doors=True)

# Calculate win percentages
stay_win_percent = (wins_stay / num_simulations) * 100
switch_win_percent = (wins_switch / num_simulations) * 100

# Print results
print(f"Wins by staying: {wins_stay} ({stay_win_percent:.2f}%)")
print(f"Wins by switching: {wins_switch} ({switch_win_percent:.2f}%)")
print("By switching doors, your chance of winning is significantly higher.")
```

EXPECTED OUTPUT:

Wins by staying: 6610 (66.10%)

Wins by switching: 6677 (66.77%)

By switching doors, your chance of winning is significantly higher.

MODEL VIVA QUESTIONS:

1. State the Monty Hall problem and explain the counter-intuitive result.
2. Why does switching doors give a $2/3$ probability of winning, while staying gives only a $1/3$ probability?
3. What is Monte Carlo simulation, and why is it used here instead of a purely theoretical/mathematical proof?
4. Explain the use of set operations (- and .pop()) in determining the revealed_door and new_door.
5. Why does the host's action of revealing a goat door provide additional information that changes the probabilities?

RESULT:

The Python program to simulate the Monty Hall problem was executed successfully, demonstrating empirically that switching doors offers a higher probability of winning than staying with the original choice, and the output was verified.